

BibleMemory: Note Passage References

Design decisions and implementation plan · October 8, 2026

Goal

Let a note be tied to the Bible passages it is about, so that grading, the notes page filter, and the review page's "notes for this passage" link find the right notes. References should be created automatically at upload, be visible and fixable by the user afterward, and later be attachable to specific highlighted text.

Decisions

Topic	Decision
Where references live	On chunks. A chunk can have several references. None is "primary".
Chunk with two passages at upload	Keep every passage the chunk is clearly about, with no cap. Do not force a single pick.
Note detail view	Collect all references from the note's chunks, group identical ones, show each once.
Grouping	Group by exact reference. "Genesis 27" and "Genesis 27:1-10" stay separate entries. No range merging.
Deleting a reference	Remove it from every chunk of the note that has it. No confirmation dialog and no "3 sections" wording. Show an Undo toast instead.
Adding a reference	Ask the AI which sections are clearly about the passage. If none match, add it to every section. No preview step. Show a short message saying what happened, with Undo.
Re-detect	A "Re-detect passages" button, run only when the user clicks it. Replaces auto-created references and leaves user-added ones alone.

Topic	Decision
Chunk regeneration	Not building it now. Nothing here blocks adding it later.
Highlight and tag (later)	The user selects text, picks passages, and a new chunk is created with exactly that text. No in-place editing: edit means delete and recreate.

Data model

References get their own table so a chunk can have any number of them. The existing columns on `note_chunks` stay in place during the switch and are dropped later.

```
CREATE TABLE chunk_refs (
  id          SERIAL PRIMARY KEY,
  user_id     INT    NOT NULL,
  note_id     INT    NOT NULL REFERENCES notes(id) ON DELETE CASCADE,
  chunk_id    INT    NOT NULL REFERENCES note_chunks(id) ON DELETE CASCADE,
  book_id     INT    NOT NULL,
  chapter     INT,
  verse_start INT,
  verse_end   INT,
  source      TEXT NOT NULL DEFAULT 'auto', -- 'auto' (tagger) or 'user'
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE NULLS NOT DISTINCT (chunk_id, book_id, chapter, verse_start,
  verse_end)
);
CREATE INDEX ON chunk_refs (chunk_id);
CREATE INDEX ON chunk_refs (book_id, chapter);
CREATE INDEX ON chunk_refs (user_id, note_id);
```

```
ALTER TABLE note_chunks ADD COLUMN source TEXT NOT NULL DEFAULT 'auto'; --
'auto' for highlight-created chunks
```

```
-- carry over the tags that exist today
INSERT INTO chunk_refs (user_id, note_id, chunk_id, book_id, chapter,
  verse_start, verse_end)
SELECT user_id, note_id, id, book_id, chapter, verse_start, verse_end
FROM note_chunks WHERE book_id IS NOT NULL;
```

Notes:

- A reference that crosses a chapter boundary (Genesis 27:30 to 28:5) is stored as one row per chapter.
- There is no cap on references per chunk. The tagger prompt asks only for passages the chunk is clearly about, which keeps a chunk from being tagged with everything.
- The note's passage list is never stored. It is always computed from its chunks' references, so there is one source of truth. Each reference row also carries `user_id` and `note_id`, copied from its chunk when it is inserted, so references can be listed and counted per user or per note without joining `note_chunks`. Count with `COUNT(DISTINCT (note_id, book_id, chapter, verse_start, verse_end))` so a reference that sits on several chunks of one note counts once.

Behaviors

1. Upload: tagging a chunk

The tagger (`matchChunkToVerse`) currently returns one reference. Change it to return a list of the passages it is clearly about, one entry per chapter, and an empty list when there are none.

- Prompt change: ask for a JSON array of `{bookId, chapter, verseStart, verseEnd}` and say to return `[]` when unclear.
- Tag once per note, not once per chunk: send all of the note's chunks (id and text) in one call, in groups of about 20 for long notes, and have it return references keyed by chunk id. Re-detect uses the same call. Insert the chunks and their references in one transaction.
- To keep multi-passage chunks rare, make chunks more focused. List-style notes (numbered or bulleted lines with no blank lines between them) should split per item in `chunkText` .

2. Note detail view: show references

One endpoint returns the consolidated list:

```
GET /notes/{id}/refs
-> [{ "bookId": 0, "book": "Genesis", "chapter": 27, "verseStart": null,
"verseEnd": null }, ...]
```

The query joins `chunk_refs` to `note_chunks`, filters by `note_id` and `user_id`, and groups by `(book_id, chapter, verse_start, verse_end)`. The UI shows each as a chip, plus an "Add passage" button and a "Re-detect passages" button. A note with no references shows "No passages yet" with the same buttons.

3. Deleting a reference

```
DELETE /notes/{id}/refs      body: { bookId, chapter, verseStart, verseEnd
}
```

Removes every matching row for that note's chunks. The response returns the deleted rows (chunk id plus reference) so the Undo toast can put them back with one request. Undo is a single `POST` of those rows. Because Undo needs the exact rows, do not rely on the grouped list alone.

4. Adding a reference

```
POST /notes/{id}/refs      body: { bookId, chapter, verseStart, verseEnd
}
-> { "mode": "matched" | "whole_note", "addedRefIds": [...] }
```

Server steps:

1. Validate the reference: the book exists, the chapter is within the book's chapter count, and `verseEnd >= verseStart`.
2. Load the passage text for that reference from the scripture repository (cap at roughly 300 words).
3. Send the passage text plus the note's chunks (id and text, each truncated) to Gemini. Ask which chunks are **clearly about** the passage, not merely related. Return JSON like `{"chunkIds": [...]}`.
4. Keep only ids that belong to this note. If the list is empty or the call fails, use every chunk of the note (`mode: whole_note`).
5. Skip chunks that already have an equal or broader reference (same book and chapter, with a null verse range or the same one).
6. Insert the rows with `source = 'user'` in one transaction using `ON CONFLICT DO NOTHING`, and return the ids of the rows actually inserted, so Undo never deletes a reference that was already there.

The UI shows one message: "Added Genesis 27 to 2 sections" or "No section matched, so Genesis 27 was added to the whole note." Both have Undo, which deletes the returned `addedRefIds`.

The prompt must be strict. A generic section like "God can still work all things for good" fits almost any passage, so a loose prompt would attach every new reference to it.

5. Re-detect passages

```
POST /notes/{id}/redetect
```

Triggered only by the button. One batched Gemini call takes all of the note's chunks and returns references per chunk. In a transaction:

- Delete `chunk_refs` rows with `source = 'auto'` for this note's chunks.
- Insert the new ones with `source = 'auto'`.
- Leave `source = 'user'` references alone, and skip chunks with `note_chunks.source = 'user'`.

This covers "everything is gone, start over" without regenerating chunks. It can bring back an auto reference the user deleted on purpose, which is why it is manual only.

6. Highlight and tag (later feature)

```
POST /notes/{id}/chunks      body: { text, refs: [...], replaceChunkId? }
DELETE /notes/chunks/{id}    (only where source = 'user')
```

- The user selects text in the note view and a popover asks for one or more passages.
- The server checks that the text really appears in the note's `raw_text` (ignoring whitespace differences) and rejects it otherwise. Never trust text sent by the client.
- It embeds the text first. If embedding fails, nothing changes. Then, in one transaction, it inserts the chunk (`source = 'user'`), inserts its references (`source = 'user'`), and deletes `replaceChunkId` if one was given.
- Edit in the UI is the same popover prefilled and saved with `replaceChunkId`. There is no update path on the server.
- Optional later: put the reference label and some verse text in front of the chunk text when embedding. This is cheap because chunks are never edited in place.

Retrieval changes

Every query that reads `book_id` or `chapter` from `note_chunks` switches to a reference check. This covers `FindRelevant`, `CountByBooks`, the vector note search, and the notes list passage filter:

```
AND EXISTS (
  SELECT 1 FROM chunk_refs r
  WHERE r.chunk_id = c.id
        AND r.book_id BETWEEN $2 AND $3
        AND ($4::int IS NULL OR r.chapter = $4)
)
```

Because an added reference can land on every chunk of a note, retrieval must stop trusting the tag alone:

- Grading should always rank eligible chunks by similarity to the passage text and keep the top few. Remove the shortcut that takes the 5 newest chunks when a user has 5 or fewer.
- Keep the recency query only as the fallback when the embedding call fails.
- Embed the passages for all ranges concurrently so grading is not slowed by one call per range.
- Cap chunks per note (about 2) and sort `source = 'user'` chunks first, so one note cannot fill every slot.

Frontend changes

- **Types and hooks:** `PassageRef`, plus hooks for listing a note's references, adding, deleting, undoing, and re-detecting. Each mutation invalidates the `["notes"]` queries.
- **Note detail view:** the references section with chips (each with a remove button), "Add passage" using the same book, chapter, and verse inputs as `RangeCard`, "Re-detect passages", and a toast with Undo.
- **Notes page filter and the review page link:** no frontend change. The list endpoint already takes `book` and `chapter`, and only its query changes.
- **Later:** selection popover and chunk chips in the note view for highlight and tag.

Implementation order

1. **Schema and switch.** Create `chunk_refs`, backfill it, and move the read queries over. Confirm that grading and the passage filter return the same notes as before. This is the risky step, so do it first with no new features mixed in.
2. **Multi-reference tagging.** Change the tagger prompt, call it once per note with all chunks, and update the chunk creation transaction. Re-upload one test note and check its rows.
3. **Always-rank retrieval.** Embed the passage, rank, cap per note, concurrent embeds, recency fallback.
4. **Detail view: read and delete.** The consolidated list endpoint, the chips, delete, and the Undo toast.
5. **Add passage.** Validation, the AI match, the whole-note fallback, and Undo.
6. **Re-detect.** The batched call and the replace-auto-only transaction.
7. **Highlight and tag.** The chunk create and delete endpoints, then the selection UI.

Steps 1 to 3 change behavior that already works, so check each with a note you know the right answer for (the Genesis 27 notes are good test data). Steps 4 to 6 are mostly new code.

Testing checklist

- After step 1: grading the Jacob and Esau passage still retrieves the same notes, and the notes page filter for Genesis 27 returns the same list as before.
- After step 2: a note that clearly touches two passages gets two reference rows on the right chunk.
- After step 3: a passage with no relevant notes retrieves nothing useful and the grader does not mention notes. A note with an added whole-note reference does not flood the prompt with unrelated chunks.
- After step 5: adding a passage with no matching section falls back to the whole note, and Undo removes exactly the rows that were added.
- After step 6: re-detect leaves user-added references untouched.

Open questions

- **Broader or equal references.** The plan skips a chunk when it already has an equal or broader reference. Check that this feels right once real notes are in use.

- **Cost of the add and re-detect calls.** Upload tagging, add, and re-detect each use one batched Gemini call per note, which is fine at current scale. For long notes, split the chunks into groups of about 20 and truncate each chunk's text before sending.
- **Untagged chunks.** If the tagger finds nothing for a chunk, it has no references and is invisible to passage filters. The detail view's "No passages yet" state and Add passage are the recovery path.
- **Chunk regeneration.** If it is ever built, it should regenerate only `source = 'auto'` chunks and re-detect only auto references, so user work survives.